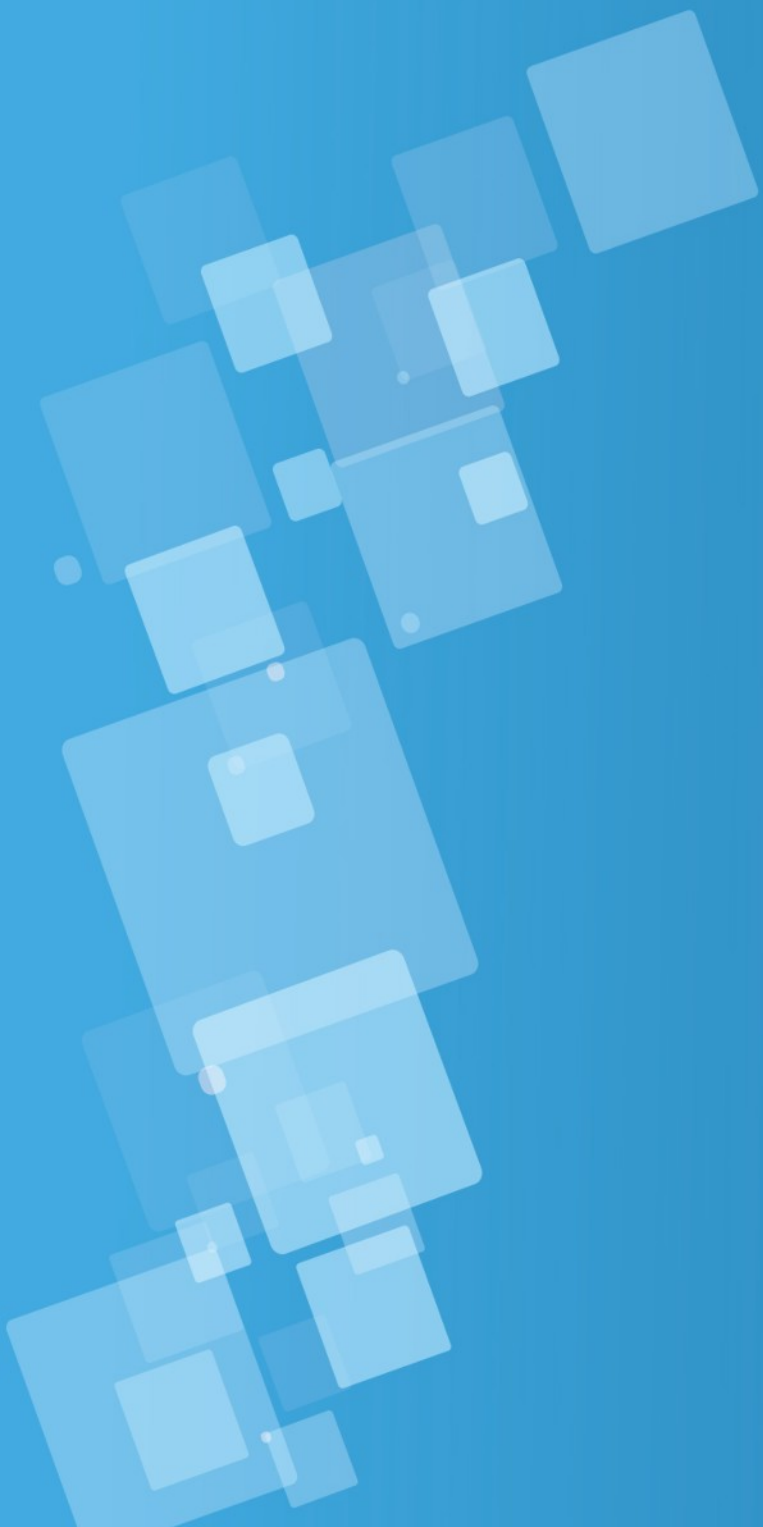




Técnicas Digitales II

RTOS (Real Time Operating System)
Sistema operativo en tiempo real
Tareas

- **Bibliografía**

Mastering the FreeRTOS™ Real Time Kernel



FreeRTOS

Es un sistema operativo en tiempo real (RTOS) diseñado específicamente para microcontroladores y pequeños microprocesadores.

- Multitarea Prioritaria (Preemptive Scheduling): El planificador (scheduler) le da el control a la tarea de mayor prioridad que esté lista para ejecutarse.
- Memoria Mínima (Micro-kernel): El núcleo está optimizado. Típicamente ocupa entre 4 KB y 9 KB de memoria Flash.
- Mecanismos de Sincronización: Incluye herramientas nativas listas para usar: Semáforos y Mutexes, Colas, Grupos de Eventos y Timers.
- Portabilidad: Está escrito casi en su totalidad en C. Con unas pocas líneas de código en lenguaje ensamblador.



Tipo de Datos

FreeRTOS utiliza notación Húngara para el nombres de las variables y funciones

Notación Húngara: anteponer un prefijo con el tipo de variable al nombre de la variable

Ejemplo: pcTaskName → p (pointer) + c (char)

Variables enteras: usa tipos de ancho fijo en lugar del tipo int genérico, esto es por portabilidad, el tamaño del int en C depende de la arquitectura.

- **int32_t**: para 32 bits con signo
- **uint32_t**: para 32 bits sin signo
- **int16_t**: para 16 bits con signo
- **uint16_t**: para 16 bits sin signo



Tipo de Datos

Además, hay dos tipos de datos adicionales en FreeRTOS (declarados en portmacro.h).

- **TickType_t**: Es el tipo de dato usado para almacenar el conteo de "ticks" del sistema (la medida del tiempo), según el tipo de arquitectura puede ser de 16 o 32 bits, usar 16 bits es útil para arquitecturas de 16 u 8 bits pero limitan el tiempo máximo de bloqueo para las tareas.
- **BaseType_t**: Es el tipo de dato más eficiente para la arquitectura específica del procesador. En un procesador de 32 bits, será un tipo de 32 bits. comúnmente usado para valores de retorno que indican verdadero o false (pdTRUE / pdFALSE) o rango muy pequeño de valores.



Tipo de Datos

El tipo de **char**, hay también ambigüedad en la declaración de C.

En FreeRTOS se debe explicitar el tipo de char, además para números se prefiere el uso de `uint_8_t` y `int_8_t`.

- **uint_8_t** (en lugar de unsigned char) para enteros sin signo.
- **int_8_t** (en lugar de signed char) para enteros con signo.
- La excepción es cuando se guarde caracteres ASCII, en ese caso se utiliza **char** o **char *** para punteros a cadenas:
 - (No es relevante el uso o no de signo para estos casos).



Tipo de Datos

Por la notación Húngara, al nombre de las variables se adiciona un prefijo

- **c:** para char o int8_t
- **s:** para int16_t (short)
- **l:** para int32_t (long)
- **x:** para BaseType_t y tipos de datos complejos (como estructuras, handles de tareas, colas, etc.).
- **u:** prefijo adicional para tipos unsigned (sin signo). Ej: uc para uint8_t.
- **p:** prefijo adicional para punteros. Ej: pc para char * (puntero a char).



Tipo de Datos

En el caso de las funciones es el tipo de dato que devuelven y el archivo donde se definen

- `vTaskDelay()` devuelven `void` definida en `task.c`
- `xTaskCreate()` devuelve `BaseType_t` definida en `trak.c`
- `xQueueReceive()` devuelve `BaseType_t` definida en `queue.c`
- `pvTimerGetTimerID()` devuelve puntero a `void` definida en `timers.c`

prv: prefijo para funciones privadas que solo se usan dentro de un mismo archivo.



Tipo de Datos

Las macros se escriben en mayúsculas y tienen prefijos que indican dónde se definen:

- **configUSE_PREEMPTION** definida en FreeRTOSConfig.h.
- **portMAX_DELAY** definidas en portmacro.h o portable.h
- **pdTRUE, pdPASS** definidas en projdefs.h
- **errQUEUE_FULL** definida en projdefs.h.



Tipo de Datos

Las macros mas comunes utilizadas en todo el FreeRTOS es

- `pdTRUE` 1
- `pdFALSE` 0
- `pdPASS` 1
- `pdFAIL` 0



¿Qué es una Tarea?

En FreeRTOS una tarea se puede resumir en una función en C que no retorna nunca.

- Normalmente se implementa como un bucle infinito `for(;;)` o `while(1)`.
- Tiene su propio stack (pila de memoria) para sus variables locales y su contexto de ejecución.
- Prototipo Requerido: Toda función de tarea debe tener este formato:

```
void mitarea(void *pvParameters);
```

¿Qué es una Tarea?

El formato de una tarea será entonces el siguiente

```
void mitarea(void *pvParameters) {  
    int32_t lVariable=0;  
    for(;;) {  
        /* código para implementar la tarea */  
    }  
    vTaskDelete(NULL);  
}
```

- lVariable es una variable local, cada instancia de la tarea crea una nueva variable local, al menos que se declare como static
- El for(;;) marca el inicio del bucle infinito, dentro de este bucle se implementa la tarea
- **El bucle no debería terminar nunca**, en caso de hacerlo se debe borrar la tarea inmediatamente con vTaskDelete(NULL)



Estados de una Tarea (Visión Simple)

En un procesador de un solo núcleo, solo una tarea puede ejecutarse a la vez

Estados Principales (modelo simplificado):

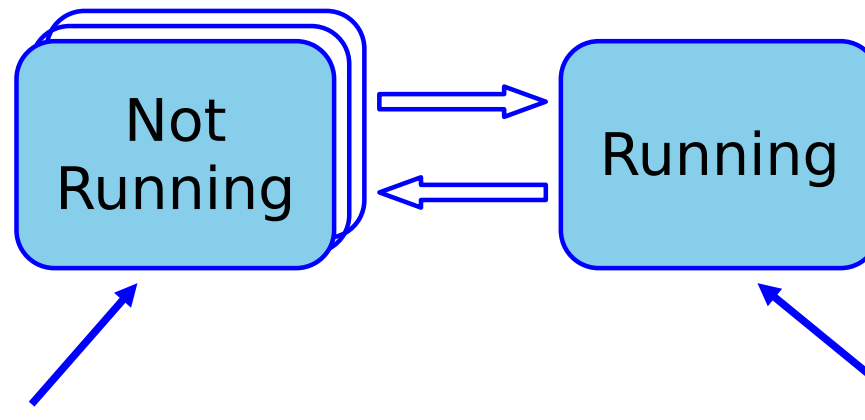
Running (En Ejecución): La tarea que tiene el control de la CPU en este instante. Solo puede haber una.

Not Running (No en Ejecución): Todas las demás tareas que están esperando su turno.

Estados de una Tarea (Visión Simple)

El Scheduler es el único que puede mover una tarea entre estados.

- Pasar de Not Running a Running se llama "switched in".
- Pasar de Running a Not Running se llama "switched out".



Todas las tareas que no están corriendo están en **Not Running**

Solo puede haber una tarea en estado **Running**



El API para Crear Tareas

Cada tarea debe ser registrada en FreeRTOS como tal, para eso usamos

`xTaskCreate()`



Registra una función como tarea para que el *scheduler* la gestione

- Esta es una de las funciones mas complicadas del FreeRTOS pero fundamental para el funcionamiento

El API xTaskCreate()

Los parámetros de xTaskCreate() son:

```
xTaskCreate(  
    Mitarea,           // (TaskFunction_t) Puntero a la función de la tarea  
    "Mi tarea",        // (const char *) Nombre (para depuración)  
    1000,              // (uint16_t) Tamaño del Stack (en palabras)  
    NULL,              // (void *) Parámetro para la tarea (opcional)  
    1,                 // (UbaseType_t) Prioridad (0 es la más baja)  
    NULL               // (TaskHandle_t) Handle de la tarea (opcional)  
);
```

- **Stack:** Es la memoria que la tarea tiene para sus variables, se expresa en word.
Ojo un stack pequeño puede generar un (stack overflow)
- **Prioridad:** Le indica al *scheduler* cuan importante es la tarea.
0 es la mas baja prioridad

El API xTaskCreate() - Valor de Retorno

xTaskCreate() devuelve un valor para indicar si tuvo éxito:

```
xTaskCreate(  
    Mitarea,           // (TaskFunction_t) Puntero a la función de la tarea  
    "Mi tarea",        // (const char *) Nombre (para depuración)  
    1000,              // (uint16_t) Tamaño del Stack (en palabras)  
    NULL,              // (void *) Parámetro para la tarea (opcional)  
    1,                 // (UbaseType_t) Prioridad (0 es la más baja)  
    NULL               // (TaskHandle_t) Handle de la tarea (opcional)  
);
```

La función puede devolver:

- **pdPASS** la tarea fue creada correctamente
- **pdFAIL** la tarea no se creo, la falla mas común es falta de memoria dinámica (heap)

Siempre se debe comprobar el valor de retorno

Ejemplo 1 - Creando dos Tareas

Se crean 2 tareas idénticas que imprime un mensaje periódicamente

Se crea un retardo ineficiente mediante un bucle

```
void vTask1( void *pvParameters ) {
    const char *pcTaskName = "Task 1 is running\r\n";
    volatile uint32_t ul; /* volatile asegura no ser optimizada */
    for( ;; ) {
        vPrintString( pcTaskName );
        /* retardo por bucle */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ );
    }
}
```

```
void vTask2( void *pvParameters ) {
    const char *pcTaskName = "Task 2 is running\r\n";
    volatile uint32_t ul; /* volatile asegura no ser optimizada */
    for( ;; ) {
        vPrintString( pcTaskName );
        /* retardo por bucle */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ );
    }
}
```

Ejemplo 1 - Creando dos Tareas

Ahora el main() crea las tareas, ambas con la misma prioridad e inicia el scheduler

```
int main( void ) {
    xTaskCreate(
        vTask1,      /* puntero a la tarea1. */
        "Task 1",    /* texto solo para debug */
        1000,        /* Stack */
        NULL,         /* sin parámetros */
        1,            /* prioridad 1. */
        NULL );       /* sin handle. */
    /* creamos otra tarea de igual prioridad */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );
    /* iniciamos el scheduler */
    vTaskStartScheduler();
    /* si todo está bien nunca se llega aquí */
    for( ;; );
}
```

El resultado mostrado es artificial por ser un simulador pero da la idea de que las funciones se ejecutan en el mismo tiempo.

Como ambas tareas tienen la misma prioridad, el scheduler las alterna en cada "tick" del sistema para que compartan el tiempo de la CPU. A esto se le llama División de tiempo o *Time Slicing*

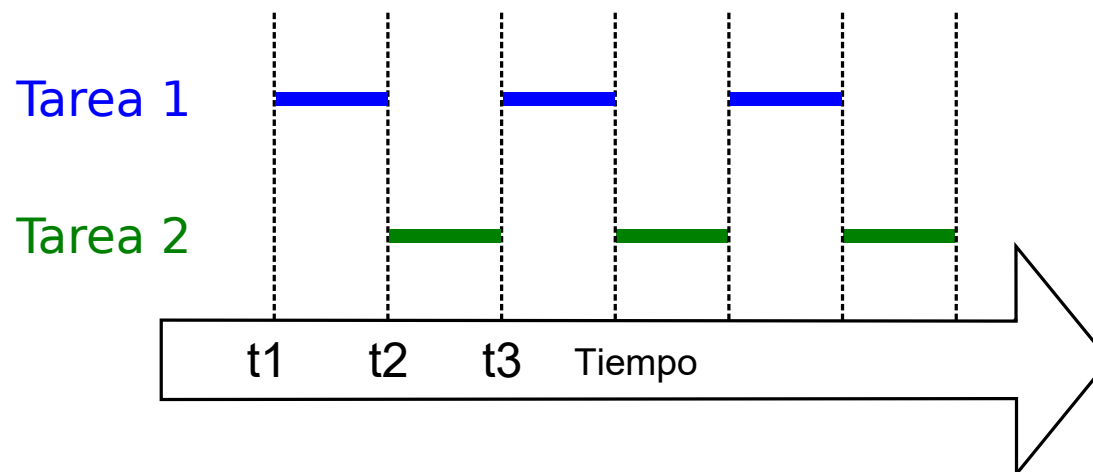
[illegible]

Ejemplo 1 - Patrón de Ejecución

Una sola tarea se puede ejecutar a la vez.

- **t1** la Tarea 1 entra en estado *Running*.
- **t2** la Tarea 1 es pausada (entra en *Not Running*), Tarea 2 entra en *Running*
- **t3** la Tarea 2 es pausada, Tarea 1 vuelve a entrar en *Running*. (en el estado que quedo en t2

El proceso se repite creando la ilusión de ejecución paralela



Ejemplo: tarea que llama tarea

Otra opción para crear la instancia de una tarea, es hacerla dentro de otra.

```
void vTask1( void *pvParameters ) {  
    const char *pcTaskName = "Tarea 1 esta ejec.\r\n";  
    volatile uint32_t ul; /* volatile asegura no ser optimizada */  
  
    /* se llama a la tarea 2 dentro de la tarea 1 antes del for(;;)*/  
    xTaskCreate( vTask2, "Tarea 2", 1000, NULL, 1, NULL );  
  
    for( ;; ) {  
        vPrintString( pcTaskName );  
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ ) {  
        }  
    }  
}
```

El resultado se verá de manera similar al Ejemplo 1

Ejemplo 2 - Reutilizando Código

En el ejemplo 1, las tareas solo cambia la cadena que imprime.

- Usaremos una sola función de tarea para crear múltiples tareas, y usamos pvParameters para enviarle las cadenas a imprimir.

```
void vTaskFunction( void *pvParameters ) {
    char *pcTaskName;
    volatile uint32_t ul;
    /* la cadena a imprimir se pasa por parámetro y se convierte
    a char (realiza un cast) */
    pcTaskName = (char *)pvParameters;

    for( ;; ) {
        vPrintString( pcTaskName ); // se imprime el parámetro
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ ) {
        }
    }
}
```

Ejemplo 2 - Reutilizando Código - main()

Ahora se realiza dos instancias de la misma función.

Estas operarán de manera independiente

```
/*se definen las cadenas como const para que se guarde en el stack*/
static const char *pcTForTask1 = "Task 1 is running\r\n";
static const char *pcTForTask2 = "Task 2 is running\r\n";
int main( void )
{
    /* se crean las 2 tareas y se pasan como parámetros las cadenas */
    xTaskCreate(vTaskFunction, "Tarea1", 1000, (void*)pcTForTask1, 1, NULL);
    xTaskCreate(vTaskFunction, "Tarea2", 1000, (void*)pcTForTask2, 1, NULL);

    vtaskStartScheduler();

    for( ;; );
}
```



Prioridades y Scheduling

No todas las tareas son iguales

- La prioridad es un número que indica la importancia de una tarea.
- La prioridad 0 es la más baja.
- El máximo se define con configMAX_PRIORITIES en FreeRTOSConfig.h
- El scheduler SIEMPRE seleccionará la tarea de mayor prioridad que esté lista para ejecutarse



Prioridades y Scheduling

FreeRTOS implementa dos tipos de Scheduling

- **Método Genérico** Escrito en C.
 - Portable y se puede usar en todas las arquitecturas de procesador, opción por defecto.
 - Mas lento y no tiene limites de prioridades.

`configUSE_PORT_OPTIMISED_TASK_SELECTION = 0`

- **Método Optimizado por Arquitectura** Escrito en ensamblador (aprovechar instrucciones de hardware).
 - No disponible en todas las arquitecturas
 - Más rápido pero se limita a 32 prioridades.

`configUSE_PORT_OPTIMISED_TASK_SELECTION = 1`

Elección: Depende de la necesidad de utilizar o no el método optimizado y si está implementado en la arquitectura usada.



El Tick del Sistema y el Tiempo

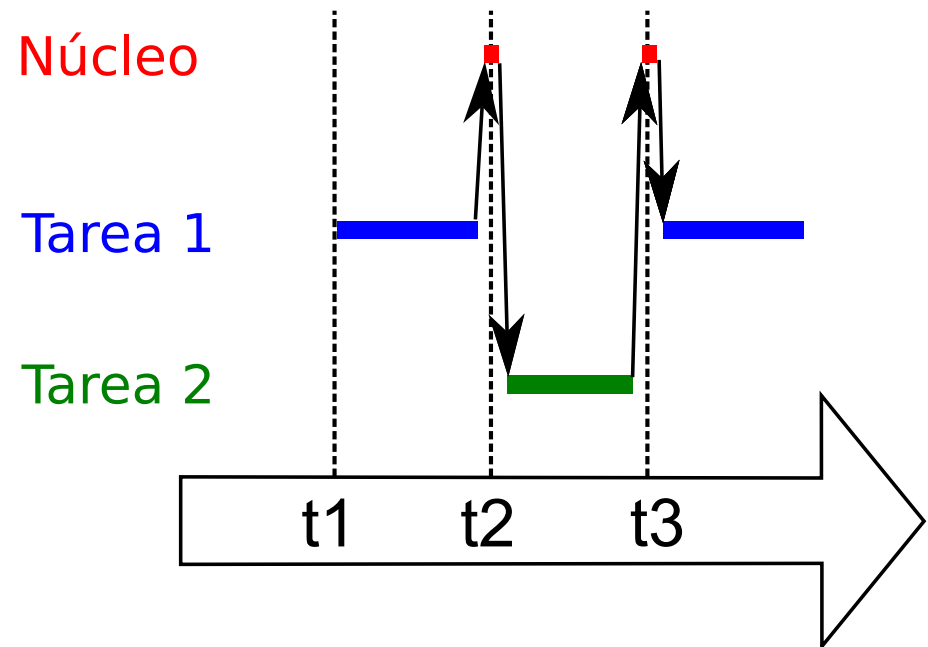
El kernel se ejecuta periódicamente gracias a una interrupción de hardware llamada ***tick interrupt***.

Esto permite seleccionar la próxima tarea a ejecutar

- La frecuencia se define con `configTICK_RATE_HZ` (ej. 100 Hz = un tick cada 10ms).
- El tiempo entre ticks se llama *Time Slice*, y es cuando el scheduler decide si debe cambiar de tarea.

Visualizando el Tick Interrupt

1. En t_1 Tarea 1 (Task 1) está en *Running*.
2. En t_2 ocurre el *Tick Interrupt*. El control pasa al kernel.
3. El *kernel* ejecuta el *scheduler*.
4. El *scheduler* ve que Tarea 2 (misma prioridad) también está lista.
5. El *scheduler* selecciona a Tarea 2.
6. La interrupción termina y el control vuelve a la Tarea 2 (que ahora está en *Running*).





Ejemplo 3 - Tareas con Diferente Prioridad

El *scheduler* garantizará que la tarea con la prioridad más alta que pueda ejecutarse sea la seleccionada para entrar en el estado "*Running*".

En el ejemplo anterior se crearon dos tareas de la misma prioridad.

¿ Qué ocurre si cambiamos la prioridad de una de las tareas ?

- Creamos entonces dos tareas.
 - Task 1 con prioridad 1.
 - Task 2 con prioridad 2.
- Ambas usan el bucle for para el retardo

Ejemplo 3 - Código de main()

- Las tareas son iguales al ejemplo anterior
- el main() queda como sigue:

```
static const char *pcTForTask1 = "Task 1 is running\r\n";
static const char *pcTForTask2 = "Task 2 is running\r\n";
int main( void )
{
    /* Crea la primera tarea con prioridad 1.*/
    xTaskCreate(vTaskFunction, "Tarea1", 1000, (void*)pcTForTask1, 1, NULL);
    /* Crea la segunda tarea con prioridad 2, (mayor prioridad que 1)*/
    xTaskCreate(vTaskFunction, "Tarea2", 1000, (void*)pcTForTask2, 2, NULL);
    /* Inicia el scheduler y las tareas comienzen a ejecutarse.*/
    vtaskStartScheduler();

    for( ;; );
}
```

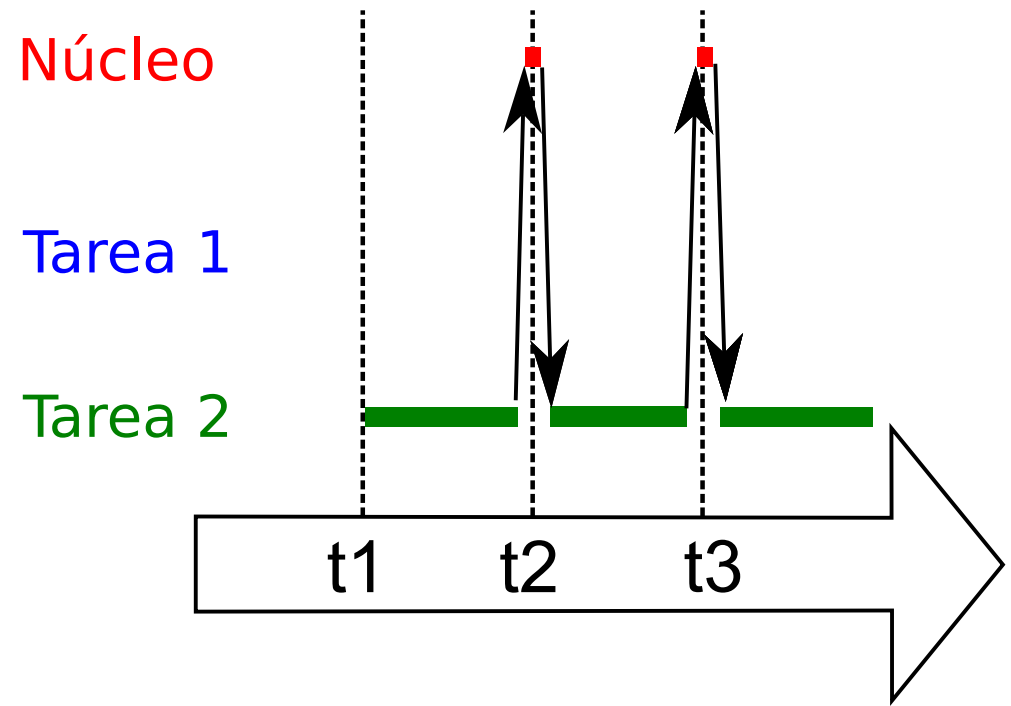
Prioridades

- ## Ejemplo 3 - Salida y Análisis
- Solo se imprime el mensaje de "Task 2" y "Task 1" nunca se ejecuta
 - Task 2 tiene mayor prioridad y, como su bucle de espera la mantiene siempre en estado Ready, nunca le da la oportunidad al scheduler de ejecutar a Task 1.
 - La tarea de baja prioridad esta "starved" o privada de recursos

[illegible]

Ejemplo 3 - Patrón de Ejecución

1. El *scheduler* arranca y elige a *Task 2* (la de mayor prioridad).
2. *Task 2* entra en su bucle for. Sigue en estado *Running*.
(siempre está haciendo algo)
3. Ocurre un *Tick Interrupt*. El *scheduler* se ejecuta, mira las tareas listas y vuelve a elegir a *Task 2* porque sigue siendo la de mayor prioridad.
4. *Task 1* nunca tiene la oportunidad de ejecutarse.





El Problema del Polling y la Espera Activa

- Las tareas creadas hasta ahora, siempre hacían algo, o imprimían un mensaje o ejecutaban un bucle. (100% del tiempo de CPU)
- La espera con un bucle es una espera activa (busy-waiting o polling) y es extremadamente perjudicial cuando se usan prioridades.
- Solo se pueden ejecutar con prioridades muy bajas, tareas con prioridad mas baja que ellas, nunca se ejecutará

Necesitamos una forma de decirle al scheduler: "No tengo nada que hacer por ahora, puedes ejecutar otra cosa"



Introducción a los Estados Extendidos

- Para que tareas de diferente prioridades puedan funcionar se deben reescribir basadas en **eventos**
- Las tareas solo tienen “algo que hacer”, luego de que ocurra un evento.
- Si el scheduler no puede seleccionar una tarea de alta prioridad porque esta no tiene nada que hacer, deberá seleccionar una de menor prioridad.
- Las tareas de alta prioridad no ocupan todo el tiempo de CPU, entonces las tareas de menor prioridad pueden ejecutarse.



Introducción a los Estados Extendidos

Vemos entonces que el estado Not Running, tiene en realidad varios sub-estados:

- **Blocked** (Bloqueado): La tarea está esperando un evento
 - **Evento temporales:** ejemplo (que pasen 10 milisegundos)
 - **Evento de sincronización:** se origina por otra tarea o por una interrupción, ejemplo (los datos lleguen a una cola)

Este tipo de sub-estado, involucra una serie de herramientas como semáforos, notificaciones directas, mutex, etc

La tarea puede bloquearse por los dos tipo de evento simultáneamente, ejemplo (que pase 10 milisegundos o que se complete la cola de datos)



Introducción a los Estados Extendidos

Vemos entonces que el estado Not Running, tiene en realidad varios sub-estados:

- **Blocked** (Bloqueado)
- **Suspended** (Suspendido): La tarea está "congelada" y no será ejecutada por el scheduler hasta que se reanude explícitamente.
No muy utilizado, pero permite poner en suspendido una tarea `vTaskSuspend()` y luego devuelta con `vTaskResume()`.



Introducción a los Estados Extendidos

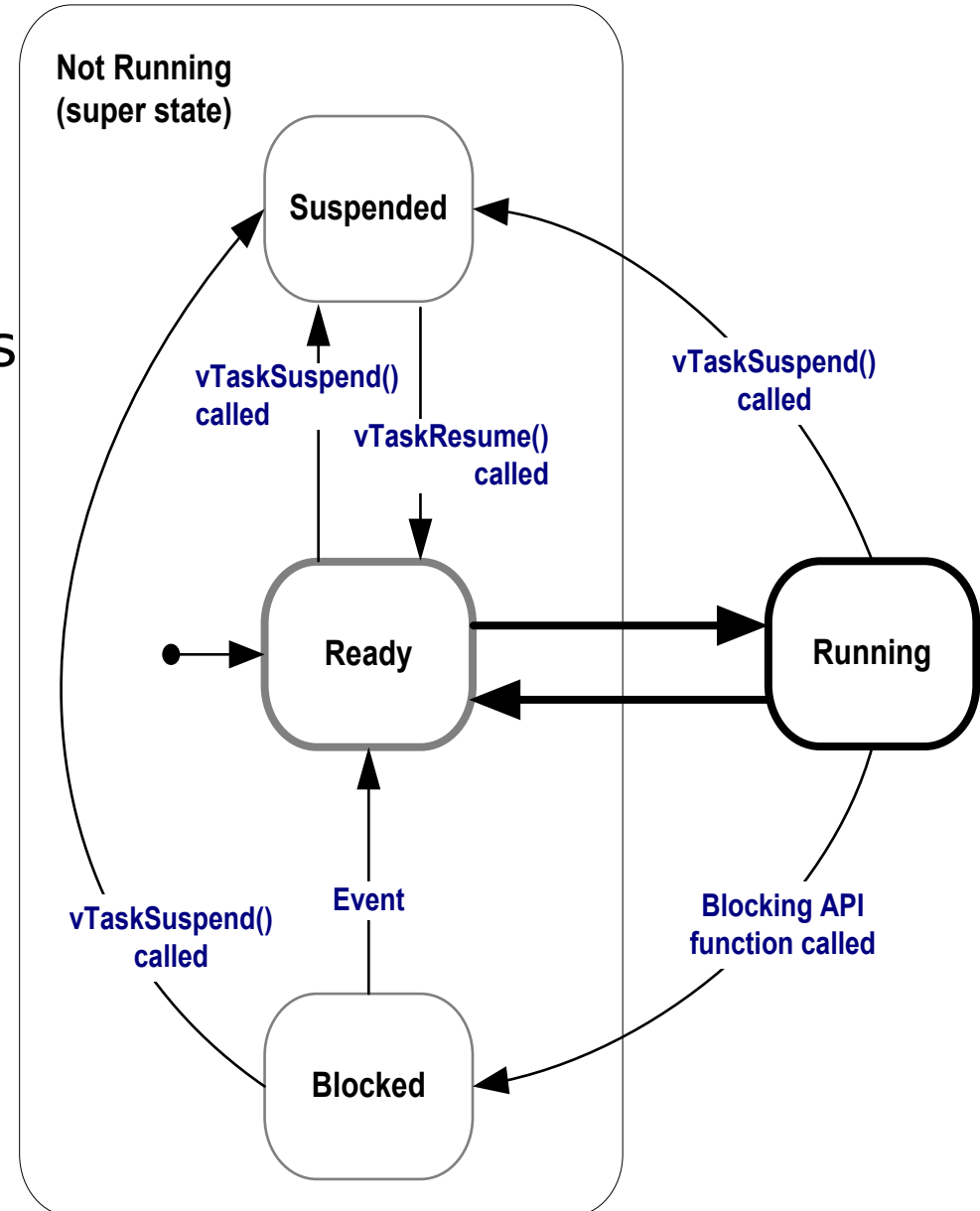
Vemos entonces que el estado Not Running, tiene en realidad varios sub-estados:

- **Blocked** (Bloqueado)
- **Suspended** (Suspendido)
- **Ready** (Listo): La tarea podría ejecutarse, pero una de mayor prioridad lo está haciendo.

Es la "sala de espera" de las tareas que tienen trabajo por hacer.

Introducción a los Estados Extendidos

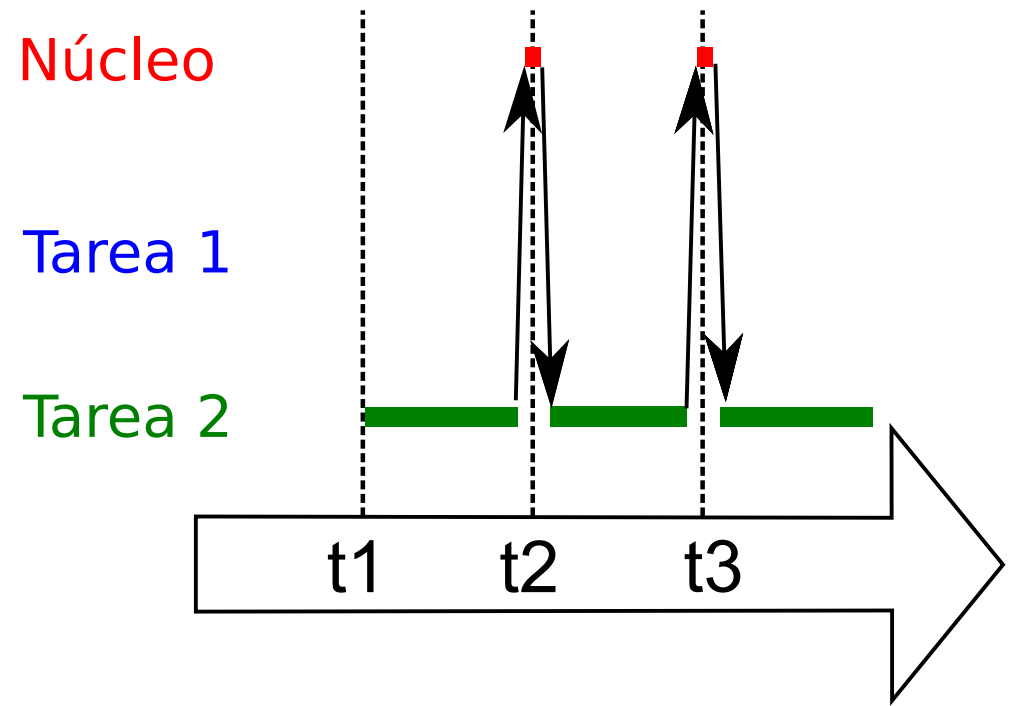
- Ahora el diagrama de estados incluye dentro del superestado **Not running** una serie de subestados
- Hasta ahora en los ejemplos, estos subestados no se utilizaron
 - Se pasaban de un estado **Ready** a **Running**, sin utilizar los estados **Blocked** o **Suspended**



Repaso del Problema

Recordemos el Ejemplo 3: una tarea de alta prioridad usando un bucle de espera activa, impide que una tarea de baja prioridad se ejecute "*starved*".

La tarea de alta prioridad está siempre en estado Ready, monopolizando la CPU. (Diseño ineficiente y peligroso)





El Estado "Blocked" (Bloqueado)

Solución: Utilizar el Estado "Blocked"

- Una tarea que está esperando un evento se encuentra en el estado **Blocked**.
- Una tarea en estado **Blocked** no es elegible para ser ejecutada por el *scheduler* y, por lo tanto, consume 0% de CPU.
- Esto permite que tareas de menor prioridad puedan ejecutarse.



Tipos de Eventos que Bloquean una Tarea

Vimos también que los eventos que bloqueaban una tarea podían ser

- **Eventos Temporales:** La tarea espera a que pase una cantidad de tiempo.
 - Ejemplo: "Despiértame en 10 milisegundos".
- **Eventos de Sincronización:** La tarea espera una señal de otra tarea o de una interrupción.
 - Ejemplo: "Despiértame cuando llegue un dato por el puerto serie".



Cómo hacer un Delays Eficiente

La función de FreeRTOS para realizar un delay es `vTaskDelay()`

- Reemplaza los bucles de espera activa.
- Pone a la tarea que la llama en estado ***Blocked*** por un número específico de "ticks" del sistema.
- Prototipo: `void vTaskDelay(TickType_t xTicksToDelay);`
- Se usa la macro `pdMS_TO_TICKS()` para especificar el tiempo en milisegundos.

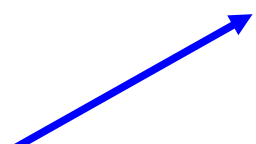
Macro `pdMS_TO_TICKS()`

Todas las funciones de FreeRTOS que esperan tiempo, lo hacen en "ticks", no en milisegundos.

Si queremos expresar los tiempos en milisegundos y hacer el código portable, usamos el macro `pdMS_TO_TICKS()`

Ejemplo: la función `vTaskDelay()` recibe el tiempo en ticks para calcular un retardo.

```
vTaskDelay(pdMS_TO_TICKS(100)); //Pausa 100 milisegundos
```



La macro devolverá el equivalente en ticks de 100 ms

Ojo!! Esta macro funciona hasta un ticks de 1kHz

Ejemplo 4 - Resolviendo la Inanición "*starved*"

Reemplazamos el bucle de espera activa por vTaskDelay()

```
void vTaskFunction( void *pvParameters ) {
    char *pcTaskName;
    /* transforma los milisegundos en ticks*/
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );
    pcTaskName = ( char * ) pvParameters; /* texto de parámetro */
    for( ;; ) {
        /* imprimir el nombre de la tarea */
        vPrintString( pcTaskName );
        /* RETARDO */
        vTaskDelay( xDelay250ms );
    }
}
```

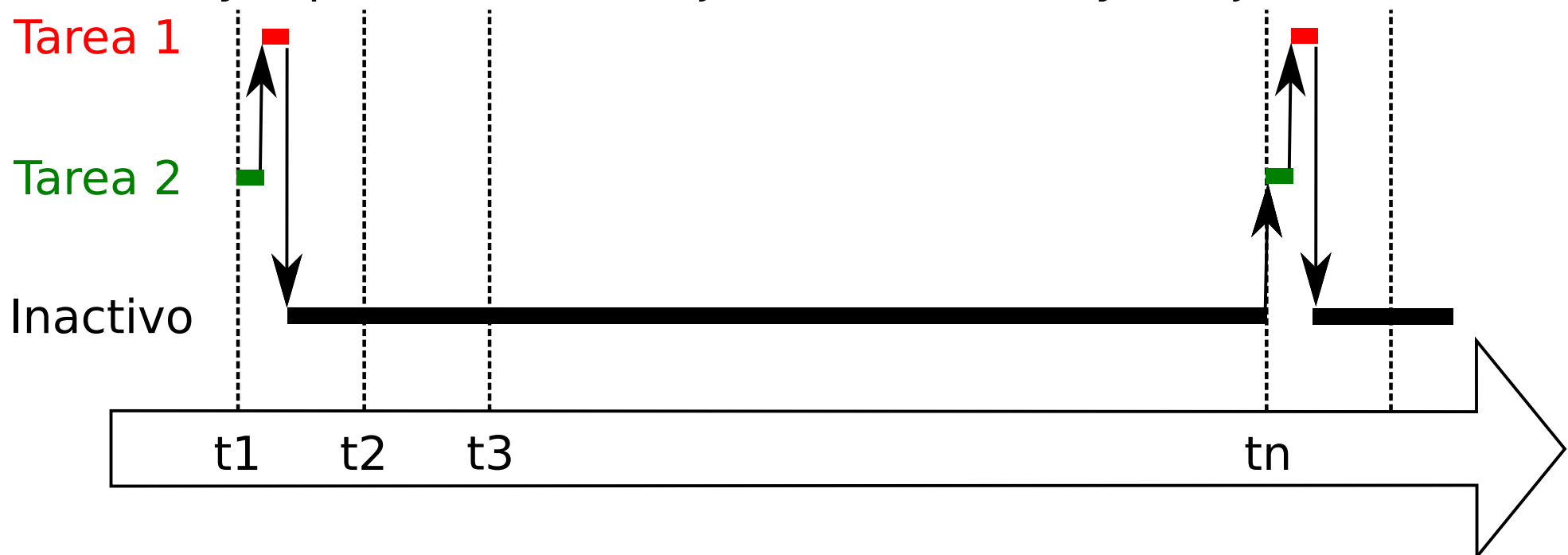
Ejemplo 4 - Salida y Análisis

- Podemos ver ahora que bajo la misma condición de prioridad (prioridad alta para tarea 2 y baja para tarea 1)
- Ambas tareas se pueden ejecutar
- Cuando la tarea de alta prioridad llama a `vTaskDelay()`, entra en estado Blocked, liberando la CPU. El scheduler entonces ejecuta la siguiente tarea de mayor prioridad que esté Ready, que es nuestra tarea de prioridad baja.

[illegible]

Ejemplo 4 - Patrón de Ejecución Detallado

- **t1** Tarea 2 (AP) se ejecuta y llama a `vTaskDelay()`. Pasa a Blocked.
- Tarea 1 (BP) ahora puede ejecutarse. Llama a `vTaskDelay()`. Entra en estado Blocked.
- Ambas bloqueadas. Se ejecuta la Tarea Idle. (Ahorro energía).
- **tn** El retardo de la Tarea 2 expira. Pasa a estado Ready, y como es la de mayor prioridad, desaloja a la Tarea Idle y se ejecuta.





Ejemplo 4 - Patrón de Ejecución Detallado

Este formato es mucho mas eficiente que el anterior

- En el retardo por bucle, la tareas ocupaban todo el tiempo, utilizando el 100% del tiempo de procesador
- En la opción con `vTaskDelay()`, la tarea se ejecuta durante un tiempo muy breve luego pasa a estado ***Blocked***, usando una pequeña fracción del tiempo del procesador.

La mayoría de las veces no hay tareas a ejecutarse (todas están en estado ***blocked*** esperando un evento)

El tiempo de procesamiento asignado a la tarea ***Idle*** es una medida de la capacidad de reserva en el sistema

Maximizar las tareas que funcionan por evento, maximiza la capacidad de reserva.



El Problema de los Delays Relativos

El `vTaskDelay()` no es perfecto para tareas periódicas

Ejemplo:

- `vTaskDelay(100)` espera 100ms desde el momento en que se llama.
- Si la tarea tarda 2ms en ejecutarse, el ciclo completo será de 102ms.

Tiempo de Ciclo = Tiempo de Ejecución + Tiempo de Delay

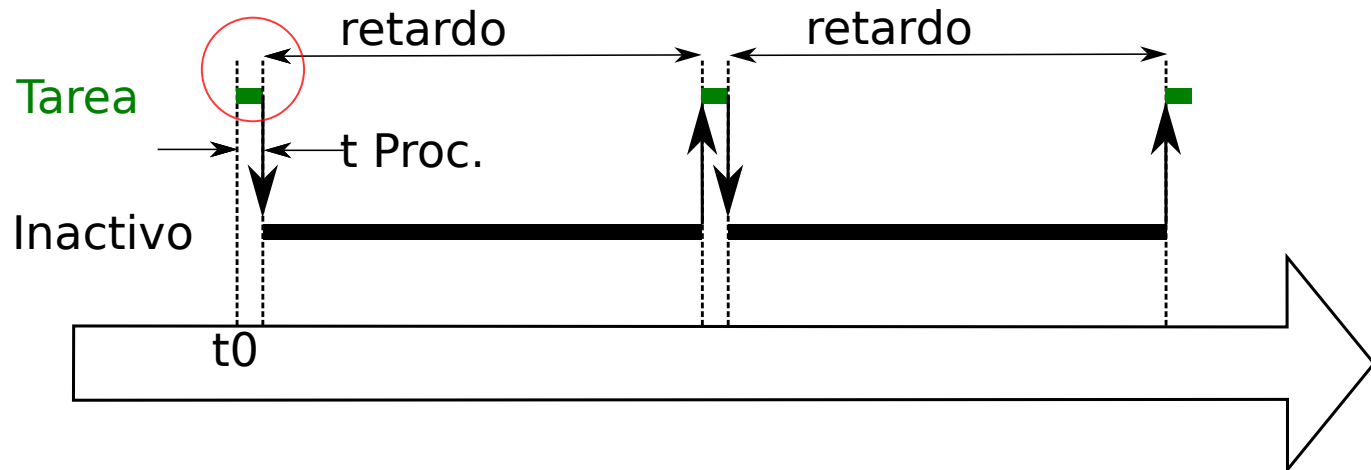
Esto introduce un deslizamiento. No es ideal para un control periódico preciso.

El API `vTaskDelayUntil()`

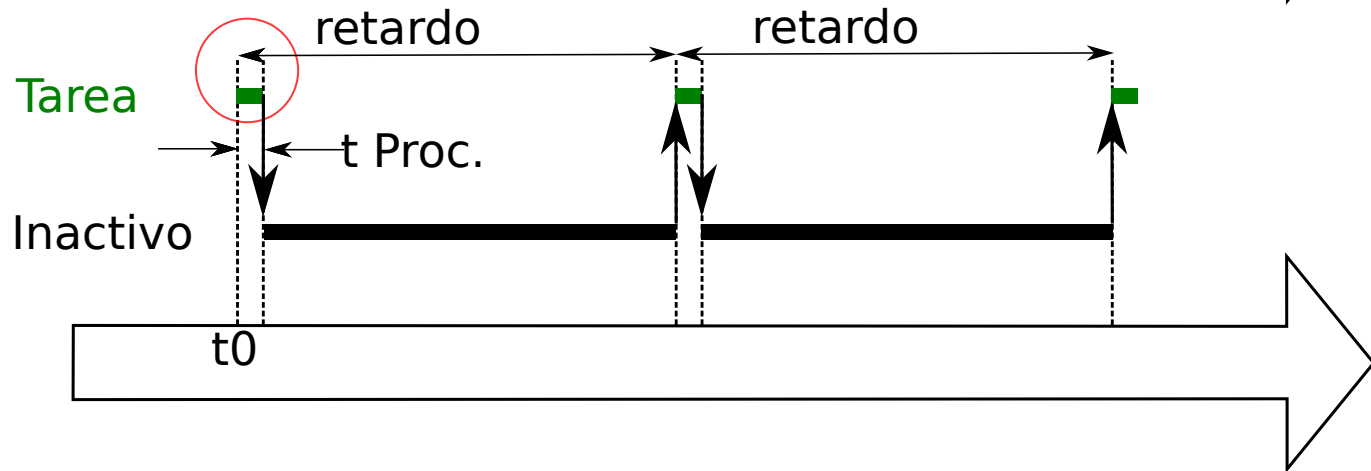
Esta función despierta a la tarea en un momento absoluto, no relativo.

Calcula el próximo tiempo de despertar basándose en el tiempo de despertar anterior.

`vTaskDelay()`



`vTaskDelayUntil()`





El API vTaskDelayUntil()

```
void vTaskDelayUntil(TickType_t *pxPreviousWakeTime,  
                    TickType_t xTimeIncrement);
```

- **pxPreviousWakeTime:** Un puntero a una variable que almacena la última vez que la tarea se despertó. Se debe inicializar una vez antes del bucle. La función la actualiza automáticamente.
- **xTimeIncrement:** La frecuencia o período fijo con el que la tarea debe ejecutarse, en ticks.

Ejemplo 5 - Código de vTaskDelayUntil()

Convertir la tarea anterior para que tenga una frecuencia de ejecución exacta.

```
void vTaskFunction( void *pvParameters ) {
    char *pcTaskName;
    TickType_t xLastWakeTime;
    pcTaskName = ( char * ) pvParameters;
    /* Inicializar xLastWakeTime con el tick actual ANTES del bucle*/
    xLastWakeTime = xTaskGetTickCount();
    for( ;; ) {
        vPrintString( pcTaskName );
        /* el Delay ejecuta cada exactamente 250 milisegundos */
        vTaskDelayUntil( &xLastWakeTime, pdMS_TO_TICKS( 250 ) );
    }
}
```

El resultado es igual al anterior, la diferencia es que ahora la tarea se ejecuta en un retardo mas exacto



Ejemplo 6 - Mezclando Tareas

Combinando Tareas **Blocked** y **No Blocked**

Veremos ahora como interactúa una tarea por bloqueo de alta prioridad con tareas de procesamiento continuo de baja prioridad.

Tenemos entonces:

- Dos tareas de baja prioridad, estas tareas.
 - Imprimen continuamente una cadena.
 - Nunca entra en **Blocked** por lo tanto siempre necesitan hacer algo "procesamiento continuo"
- Una tarea de alta prioridad periódica, esta tarea.
 - Imprime una cadena periódicamente
 - Utiliza `vTaskDelayUntil()` pasando a **Blocked** luego de cada impresión



Ejemplo 6 - Mezclando Tareas

Tarea continua de baja prioridad

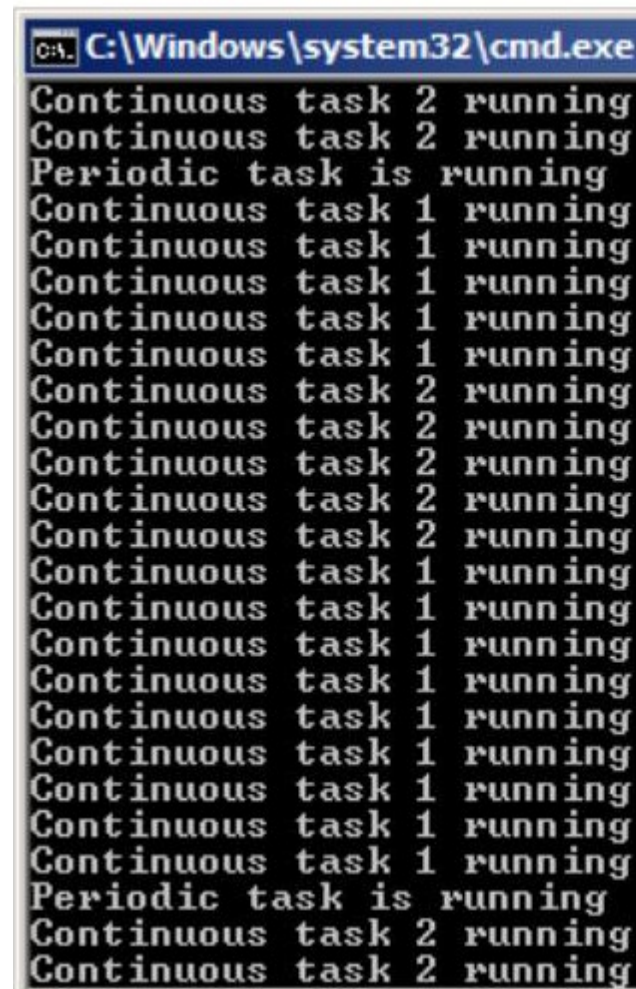
```
void vContinuousProcessingTask( void *pvParameters ) {
    char *pcTaskName;
    pcTaskName = ( char * ) pvParameters;
    for( ;; ) {
        vPrintString( pcTaskName );
    }
}
```

Tarea por eventos de alta prioridad

```
void vPeriodicTask( void *pvParameters ) {
    TickType_t xLastWakeTime;
    const TickType_t xDelay3ms = pdMS_TO_TICKS( 3 );
    xLastWakeTime = xTaskGetTickCount();
    for( ;; ) {
        vPrintString( "Periodic task is running\r\n" );
        vTaskDelayUntil( &xLastWakeTime, xDelay3ms );
    }
}
```

Ejemplo 6 - Patrón de Ejecución

- La tarea periódica (*Periodic*) se ejecuta y entra en estado **Blocked**.
- Durante ese tiempo, las tareas continuas (task1, task2) se reparten la CPU (time slicing).
- Cuando el retardo de Periodic expira, desaloja inmediatamente a la tarea. continua que se estuviera ejecutando.



A screenshot of a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The window displays a log of task execution. The log shows a sequence of "Continuous task 1 running" and "Continuous task 2 running" messages, followed by a "Periodic task is running" message. The log is as follows:

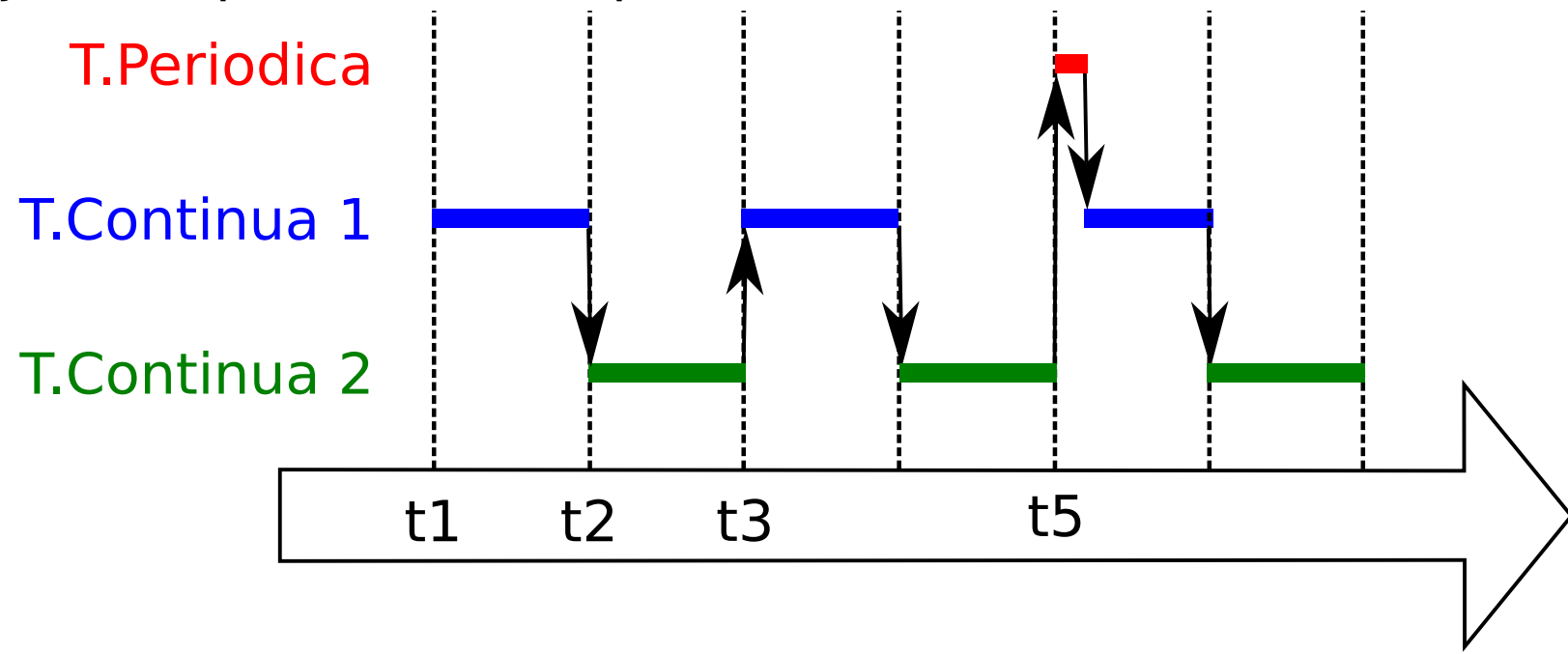
```
C:\Windows\system32\cmd.exe
Continuous task 2 running
Continuous task 2 running
Periodic task is running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 2 running
Continuous task 2 running
Continuous task 2 running
Continuous task 2 running
Continuous task 2 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Periodic task is running
Continuous task 2 running
Continuous task 2 running
```

Ejemplo 6 - Patrón de Ejecución

Cada tarea se ejecuta en un *tick* completo, como ambas tiene la misma prioridad el *scheduler* reparte el tiempo entre las dos.

En t5 la interrupción de tiempo detecta que el bloqueo de la tarea periódica expiro, como es la de mayor prioridad se ejecuta de inmediato

La tarea periódica imprime la cadena y luego entra en modo **Blocked** el *scheduler* elije una que esté en espera.





La Tarea Idle y Gestión Avanzada

¿Qué pasa cuando no hay nada que hacer?

- FreeRTOS crea automáticamente una Tarea *Idle* al iniciar el scheduler.
- Tiene la prioridad más baja posible (0) garantizando que cualquier otra tarea tiene mas prioridad para el *scheduler*, es decisión del programador asignar una tarea con prioridad 0 y compartir tiempo con Idle.
- Su propósito es asegurar que siempre haya al menos una tarea en estado Ready para ejecutarse.
- La cantidad de tiempo que se ejecuta la Tarea *Idle* es una medida directa del **tiempo de CPU libre** en tu sistema.
- **Idle** es la encargada de limpiar recursos del kernel, Si la aplicación utiliza por ejemplo `vTaskDelete()`, es importante que se ejecute de vez en cuando,



El Idle Hook

```
void vApplicationIdleHook(void);
```

Aprovechando el Tiempo Libre

- Podemos "enganchar" nuestra propia función al bucle de la Tarea Idle. Se llama Idle Hook.
- `configUSE_IDLE_HOOK = 1` en `FreeRTOSConfig.h` para que la función Idle Hook funcione
- Usos Comunes:
 - Ejecutar tareas de fondo de muy baja prioridad.
 - Poner el procesador en modo de bajo consumo
No eficaz pero sencillo de poner el procesador en modo bajo consumo.
 - Realizar mediciones o comprobaciones de estado.
Por ejemplo medir cuanto tiempo el procesador está en Idle es una medida de la capacidad ociosa.



El Idle Hook

Limitaciones

- Una función de Idle Hook nunca debe intentar bloquear o suspender.
Bloquear la tarea inactiva podría provocar que ninguna tarea disponible entre en estado de ejecución.
- Deben ser funciones rápidas, que permita volver a la función *Idle* que la llamó
Esto es porque *Idle* puede requerir hacer actividades (por ejemplo la limpieza mencionada anteriormente)

Ejemplo 7 - Implementando el Idle Hook

Usar el Idle Hook para contar cuántos ciclos de CPU están libres.

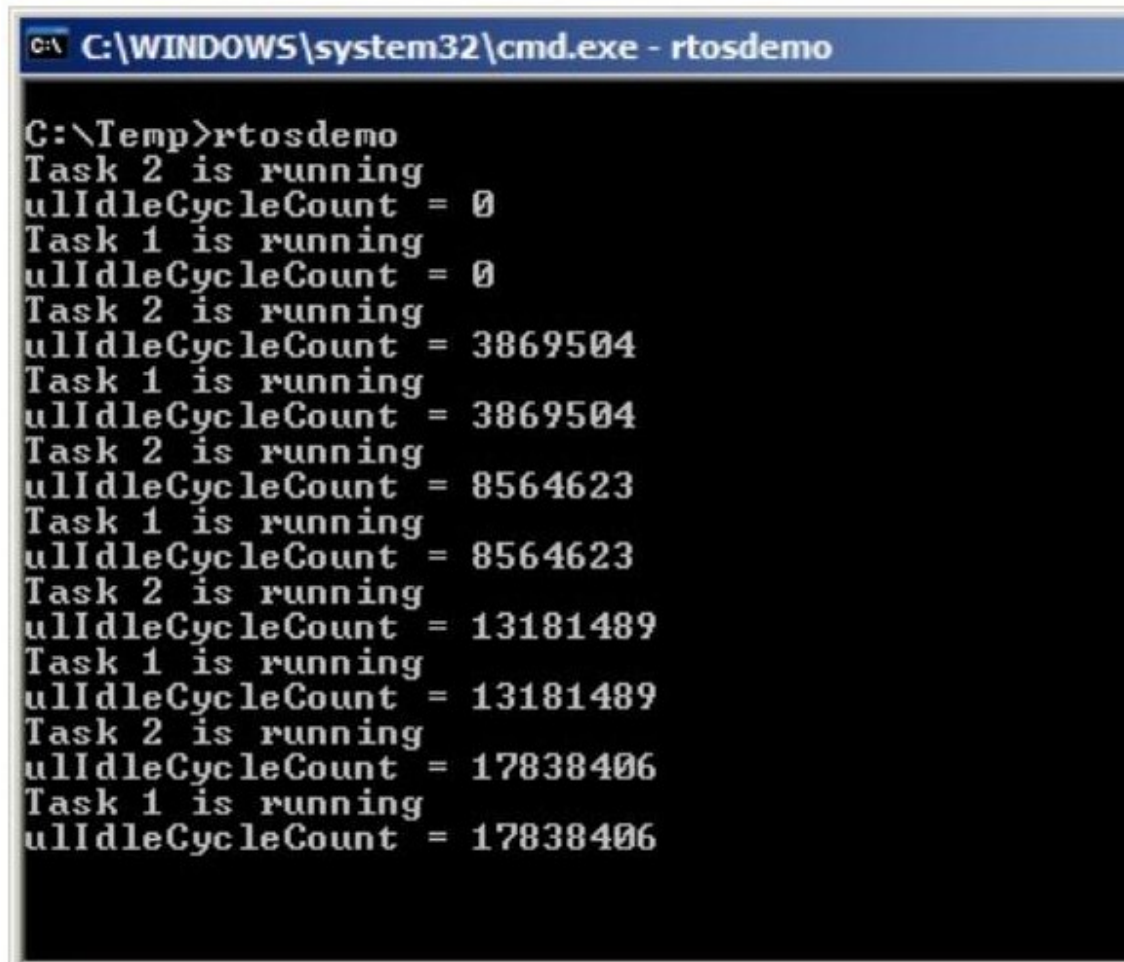
```
/* declarar la variable que será incrementada por la Idle Hook. */
volatile uint32_t ulIdleCycleCount = 0UL;
/* función Idle hook (sin parámetros y devuelve void)*/
void vApplicationIdleHook( void )
{
    /* incrementa la variable */
    ulIdleCycleCount++;
}
```

modificamos levemente la tarea principal

```
void vTaskFunction( void *pvParameters ) {
    char *pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );
    pcTaskName = ( char * ) pvParameters;
    for( ;; ) {
        /* Se imprime nombre de tarea y ulIdleCycleCount */
        vPrintStringAndNumber( pcTaskName, ulIdleCycleCount );
        vTaskDelay( xDelay250ms );
    }
}
```

Ejemplo 7 - Resultados

- El contador `ulIdleCycleCount` alcanza valores altísimos entre cada ejecución de las tareas principales.
- Esto demuestra visualmente la enorme cantidad de tiempo de CPU que se ahorra al usar retardos eficientes en lugar de espera activa.



```
C:\WINDOWS\system32\cmd.exe - rtosdemo

C:\Temp>rtosdemo
Task 2 is running
ulIdleCycleCount = 0
Task 1 is running
ulIdleCycleCount = 0
Task 2 is running
ulIdleCycleCount = 3869504
Task 1 is running
ulIdleCycleCount = 3869504
Task 2 is running
ulIdleCycleCount = 8564623
Task 1 is running
ulIdleCycleCount = 8564623
Task 2 is running
ulIdleCycleCount = 13181489
Task 1 is running
ulIdleCycleCount = 13181489
Task 2 is running
ulIdleCycleCount = 17838406
Task 1 is running
ulIdleCycleCount = 17838406
```



Cambiando Prioridades en Tiempo de Ejecución

Las Prioridades no son Estáticas

- Podemos cambiar la prioridad de una tarea mientras el sistema está en funcionamiento.
- Utilizamos
 - `vTaskPrioritySet(xTaskHandle, uxNewPriority);`
para establecer una nueva prioridad.
 - `uxTaskPriorityGet(xTaskHandle);`
para consultar la prioridad actual.
- `xTaskHandle` ("handle" de tarea) es una referencia a una tarea específica como un puntero a una tarea y se obtiene cuando se crea la misma con `xTaskCreate()`, por medio de su último parámetro.

Cambiando Prioridades en Tiempo de Ejecución

```
void vTaskPrioritySet(TaskHandle_t pxTask,  
                     UBaseType_t uxNewPriority)
```

Permite modificar las prioridades

- **pxTask** Handle de la tarea.
(el parámetro en NULL referencia a la propia tarea)
- **uxNewPriority** Limitada por la máxima prioridad disponible
(configMAX_PRIORITIES - 1),

IMPORTANTE: Se debe asignar `INCLUDE_vTaskPrioritySet = 1` en `FreeRTOSConfig.h` para poder usar esta API.



Cambiando Prioridades en Tiempo de Ejecución

```
UBaseType_t uxTaskPriorityGet(TaskHandle_t pxTask);
```

Leer la prioridad

- **pxTask** es el identificador de la tarea (handle) obtenido en `xTaskCreate()`.

IMPORTANTE: Se debe asignar `INCLUDE_uxTaskPriorityGet = 1` en `FreeRTOSConfig.h` para poder usar esta API.



Ejemplo 8 - Prioridades Dinámicas

Crear dos tareas que nunca pasan al estado ***Blocked***, en este formato, la tarea de mayor prioridad siempre estará en ejecución

Este ejemplo permitirán que las tareas "ceden" el control de la CPU mutuamente cambiando sus prioridades.

Flujo:

1. T1 (alta prioridad) se ejecuta.
2. T1 sube la prioridad de T2 por encima de la suya.
3. El *Scheduler* detiene T1 y ejecuta T2.
4. T2 baja su propia prioridad por debajo de la de T1.
5. El *Scheduler* detiene T2 y ejecuta T1.
6. El ciclo se repite.

Ejemplo 8 - Prioridades Dinámicas

```
void vTask1( void *pvParameters ) {
    UBaseType_t uxPriority;
    /* consultar prioridad de la tarea (NULL para esta tarea)*/
    uxPriority = uxTaskPriorityGet( NULL );
    for( ;; ) {
        vPrintString( "Task 1 is running\r\n" );
        vPrintString( "Aumentar prioridad de Task 2\r\n" );
        vTaskPrioritySet( xTask2Handle, ( uxPriority + 1 ) );
        /* esta parte se ejecuta cuando la tarea 2 volvió a
           bajar su prioridad */
    }
}
```

```
void vTask2( void *pvParameters ) {
    UBaseType_t uxPriority;
    uxPriority = uxTaskPriorityGet( NULL );
    for( ;; ) {
        vPrintString( "Task 2 is running\r\n" );
        vPrintString( "Bajar la prioridad de Task\r\n" );
        vTaskPrioritySet( NULL, ( uxPriority - 2 ) );
    }
}
```



Ejemplo 8 - Prioridades Dinámicas

En el main y al momento de crear las tareas, se utiliza una variable global xTask2Handle donde se guarda el handle al momento de crear la tarea 2

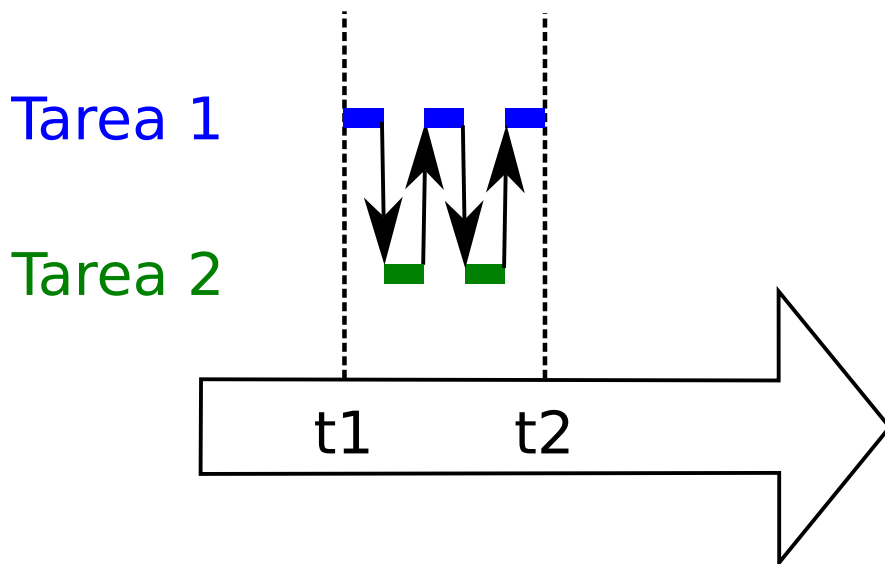
```
/* variable global que contiene el handle de Task 2. */
TaskHandle_t xTask2Handle = NULL;
int main( void ) {
    /* task 1 con prioridad 2*/
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );
    /* task 2 con prioridad 1 y guarda su handle en xTask2Handle*/
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, &xTask2Handle );

    vTaskStartScheduler();
    for( ;; );
}
```

Ejemplo 8 - Patrón de Ejecución

Visualizando el Cambio de Prioridad

El diagrama muestra claramente cómo el control de la CPU pasa de una tarea a otra, no por un tick, sino en el instante exacto en que `vTaskPrioritySet` es llamada y la tarea de mayor prioridad cambia.



```
C:\Windows\system32\cmd.exe
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
```



Eliminando Tareas

```
void vTaskDelete(TaskHandle_t xTask );
```

- Elimina una tarea.
- Una tarea puede eliminarse a sí misma pasando NULL como handle:
vTaskDelete(NULL);.
- **Importante:**
 - La Tarea *Idle* es quien libera memoria (stack y TCB)→ Si se elimina tareas, la tarea *Idle* debe ejecutarse.
 - La memoria asignada por el propio núcleo se liberará. Cualquier memoria u otro recurso de la tarea, debe ser liberado explícitamente.



Ejemplo 9 - Creación y Eliminación Dinámica

Ciclo de vida completo de una tarea: creación y eliminación en tiempo de ejecución.

1. T1 (baja prioridad) se ejecuta y crea a T2 (alta prioridad).
2. El *Scheduler* detiene T1 y ejecuta T2.
3. T2 imprime un mensaje y se auto-elimina con `vTaskDelete(xTask2Handle)`.
4. T2 ya no existe. T1 vuelve a ser la tarea de mayor prioridad Ready y se reanuda.
5. T1 entra en un `vTaskDelay()`. La Tarea Idle se ejecuta y libera la memoria de T2.

Ejemplo 9 - Creación y Eliminación Dinámica

```
TaskHandle_t xTask2Handle = NULL;
void vTask1( void *pvParameters ) {
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100UL );
    for( ;; ) {
        vPrintString("Task 1 is running\r\n" );
        /* Crea task 2 con alta prioridad y se obtiene el handle */
        xTaskCreate(vTask2, "Task 2", 1000, NULL, 2, &xTask2Handle );
        /* task 2 tendrá mas prioridad que task 1 cuando cuando pase
        por aquí task 2 ya se debe haber ejecutado */
        vTaskDelay( xDelay100ms );
    }
}
```

```
void vTask2( void *pvParameters )
{
    vPrintString("Task 2 is running and about to delete itself\r\n");
    vTaskDelete( xTask2Handle );
}
```



Ejemplo 9 - Creación y Eliminación Dinámica

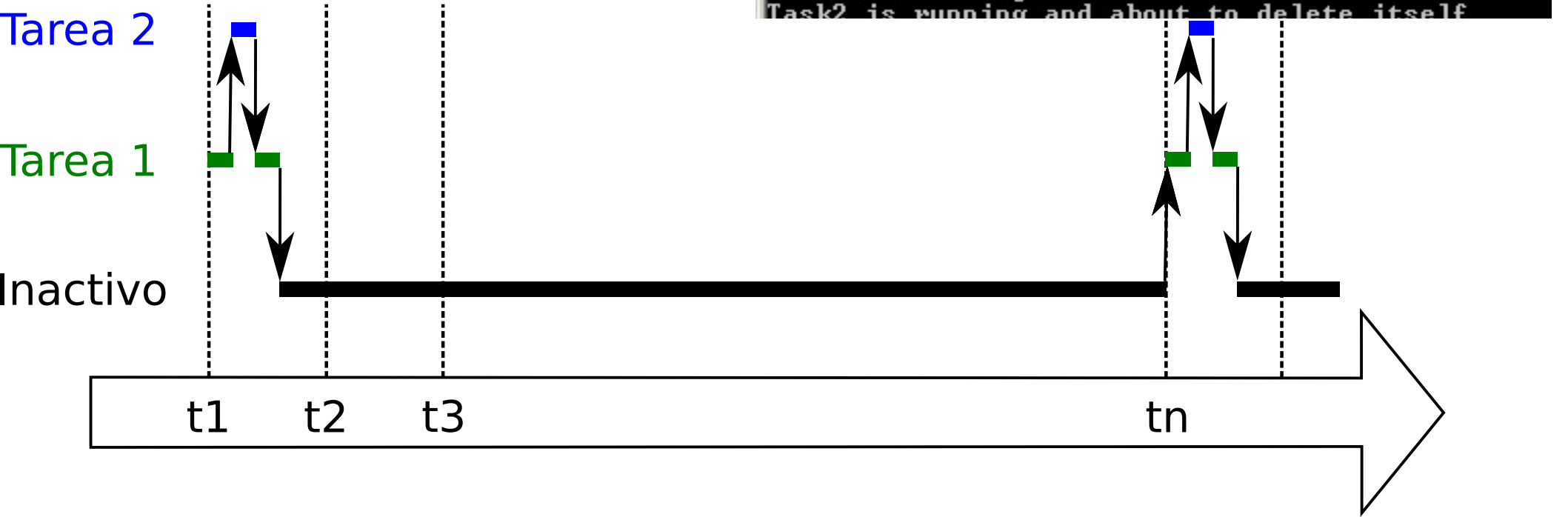
- El main() solo crea la Task 1

```
int main( void ) {  
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );  
    vTaskStartScheduler();  
    for( ;; );  
}
```

Ejemplo 9 - Patrón de Ejecución

Sigue el flujo completo, mostrando cómo la creación de una tarea de mayor prioridad causa una preemption (apropiación) inmediata y cómo, tras su eliminación, la tarea original recupera el control.

```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
```





El Estado "Suspended" (Suspendido)

El último estado es ***Suspended***.

Una tarea en este estado es invisible para el *scheduler*. No se ejecutará bajo ninguna circunstancia.

APIs:

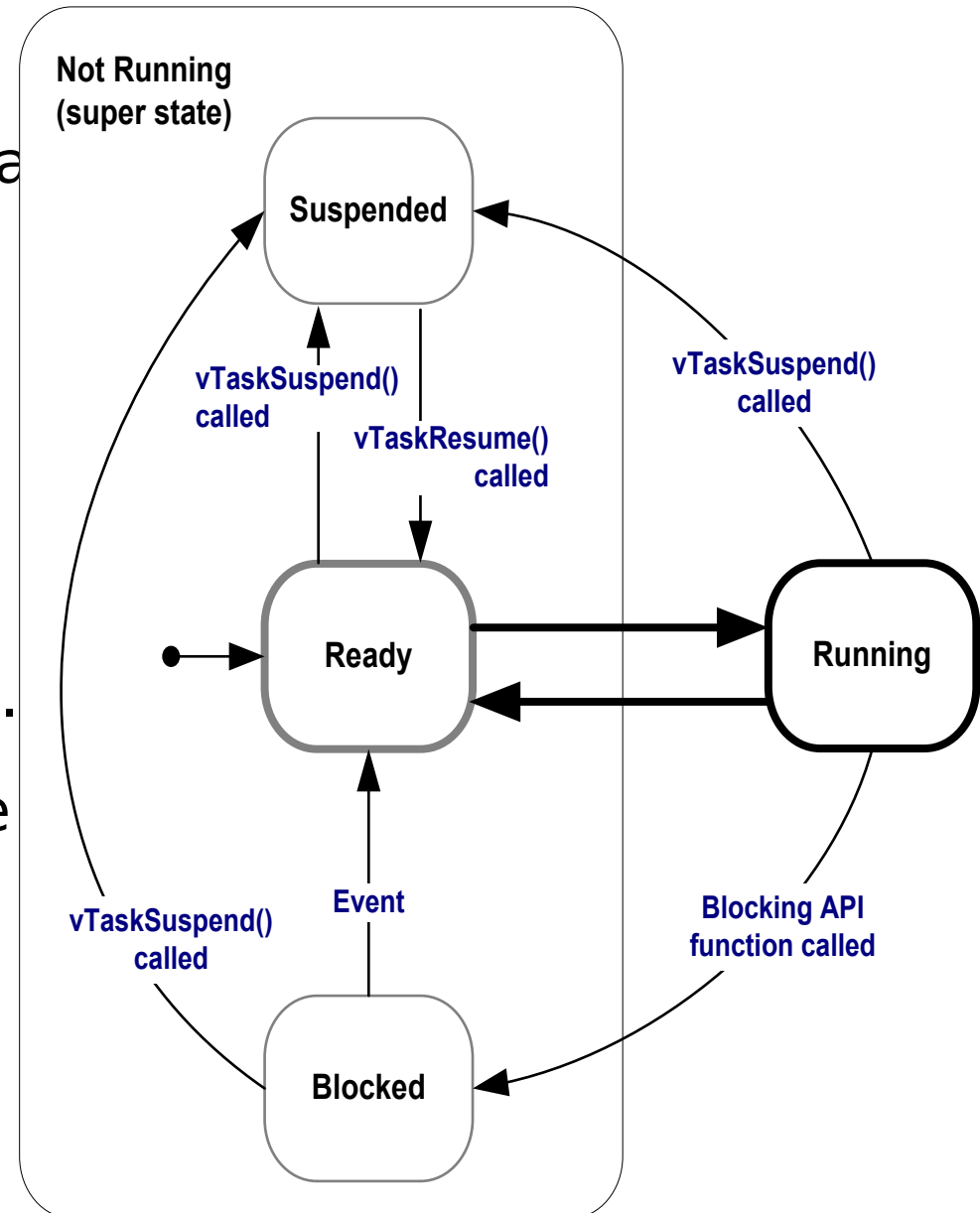
- `vTaskSuspend(xTaskHandle)`; para suspender una tarea.
- `vTaskResume(xTaskHandle)`; para reanudarla.

Su uso es menos común que el estado ***Blocked***.

Diagrama de Estados Completo

Resumen de Transiciones:

- ***Running -> Blocked***: Llamando a `vTaskDelay`, `xQueueReceive`, etc.
- ***Blocked -> Ready***: El evento esperado ocurre (expira el delay, llega un dato).
- ***Ready -> Running***: El scheduler la elige (es la de mayor prioridad).
- ***Running -> Ready***: Una tarea de mayor prioridad la desaloja.
- ***Cualquiera -> Suspended***: Llamando a `vTaskSuspend`.





¿ Preguntas ?